CS215 : Introduction to software Engineering Assignment 1

FACTUALY OF COMPUTER AND ARTIFICIAL INTELLIGENCE, CAIRO UNIVERSITY

Year 2025-2026

Second Semester
Course Instructors: Dr. Mohammad El-Ramly
Software Engineering Assignment 2 — Task 2 Draft

BudgetWise — Personal Budgeting App

TA: Ahmed Samir Section: S28

Team Members				
#	Full Name	Student ID	Email	Section
1	Ibrahem Ahmed Ezzat	20240004	20240004@stud.fci-cu.edu.eg	28
2	Mohamed Abdullah Abdulhafiz	20240515	20240515@stud.fci-cu.edu.eg	28
3	Hesham Mohamed Fathy	20240649	20240649@stud.fci-cu.edu.eg	28
4	Youssef Sayed Ali	20240692	20240692@stud.fci-cu.edu.eg	17

Table of Contents

Sec	Title	Page
1	**Task 1 — Opportunities Research**	3
1.1	Member #1 — Ibrahem Ahmed Ezzat	3
1.2	Member #2 — Mohamed Abdullah Abdulhafiz	7
1.3	Member #3 — Youssef Sayed Ali	9
1.4	Member #4 — Hesham Mohamed Fathy	12
1.5	Team Summary Table	15
2	**Task 2 — Software Design Specification (Draft)**	17
2.1	System Architecture Diagram	19
2.2	Class Diagram	21
2.3	Class Descriptions	22
2.4	Sequence Diagrams (7 Diagrams)	23
2.4.1	SD-01: User Sign-Up	23
2.4.2	SD-02: User Log-In	24
2.4.3	SD-03: Add Transaction	25
2.4.4	SD-04: Create / Edit Budget	26
2.4.5	SD-05: Budget Over-Limit Alert	27
2.4.6	SD-06: Set and Track Financial Goal	28
2.4.7	SD-07: View Reports and Analytics	29
2.5	Class-Sequence Traceability Table	30

Task 2 — Software Design Specification (Draft)

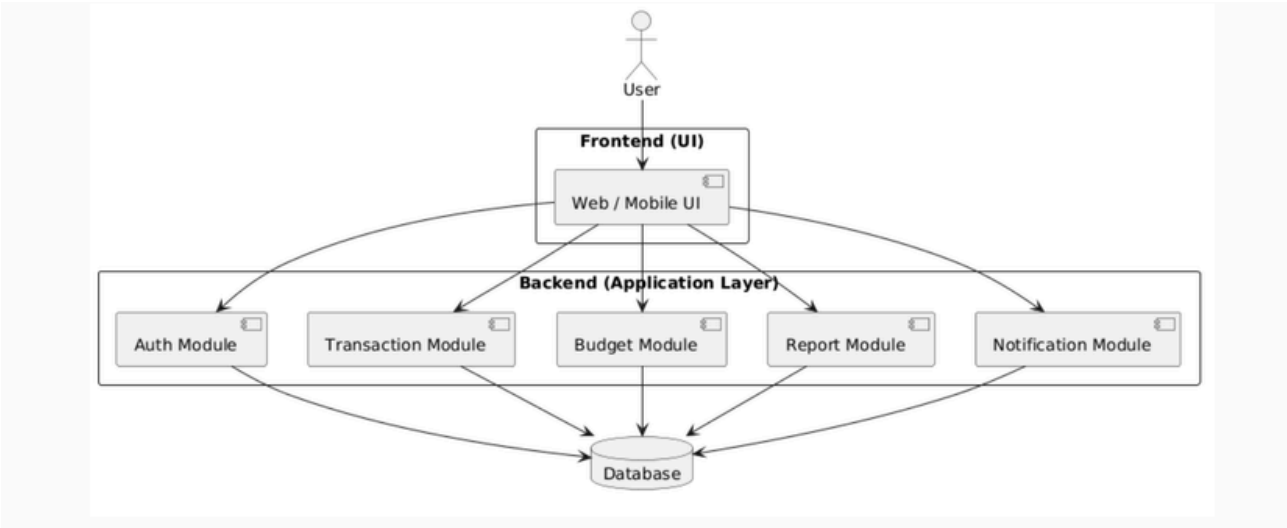
Based on the BudgetWise SRS — Personal Budgeting Software. This draft Software Design Specification (SDS) covers the system architecture, class diagram, class descriptions, and seven sequence diagrams for the most important user stories. All diagrams are generated using PlantUML. The PlantUML source code for each diagram is provided at the end of this document.

2.1 System Architecture

The BudgetWise system follows a 3-Tier MVC (Model-View-Controller) architecture. This pattern separates the application into three interconnected layers, ensuring modularity, testability, and maintainability.

Layer	Components	Responsibility
Presentation (View)	LoginScreen, DashboardScreen, TransactionScreen, BudgetScreen, GoalScreen, ReportScreen	Renders UI; captures user input; displays data from Controller
Business Logic (Controller)	AuthController, TransactionController, BudgetController, GoalController, ReportController	Processes user actions; applies business rules; coordinates between View and Model
Data / Persistence (Model + DB)	User, Transaction, Budget, Goal, Category, DatabaseManager	Stores domain data; persists to SQLite/PostgreSQL; retrieves records on demand

Architecture Diagram



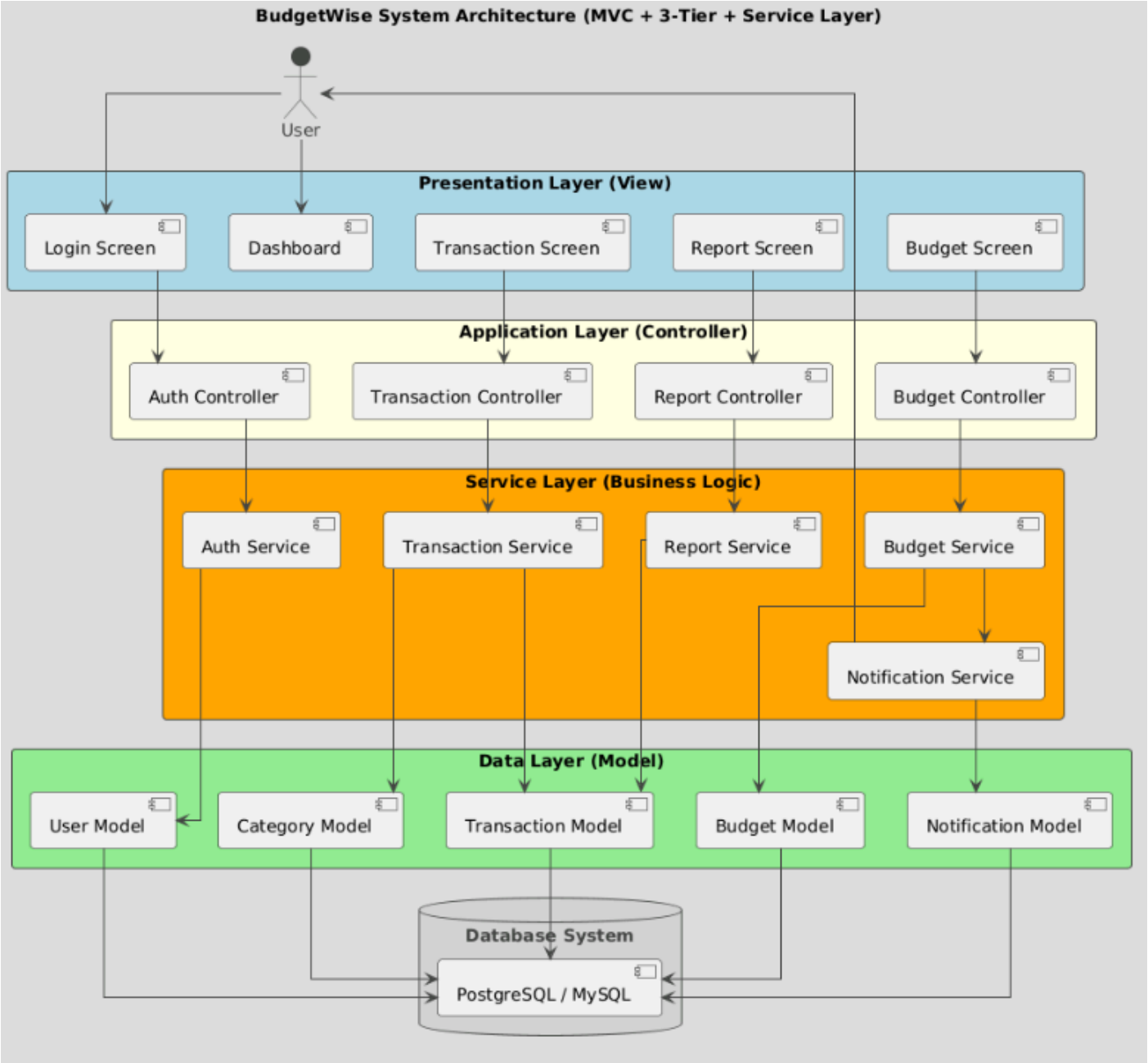


Figure A1: BudgetWise 3-Tier MVC

Architecture Diagram PlantUML Code: See

Section 2.5 — Code Block A1

2.2 Class Diagram

The class diagram below captures all domain, boundary, and control classes of the BudgetWise system, organised into three packages corresponding to the MVC layers.

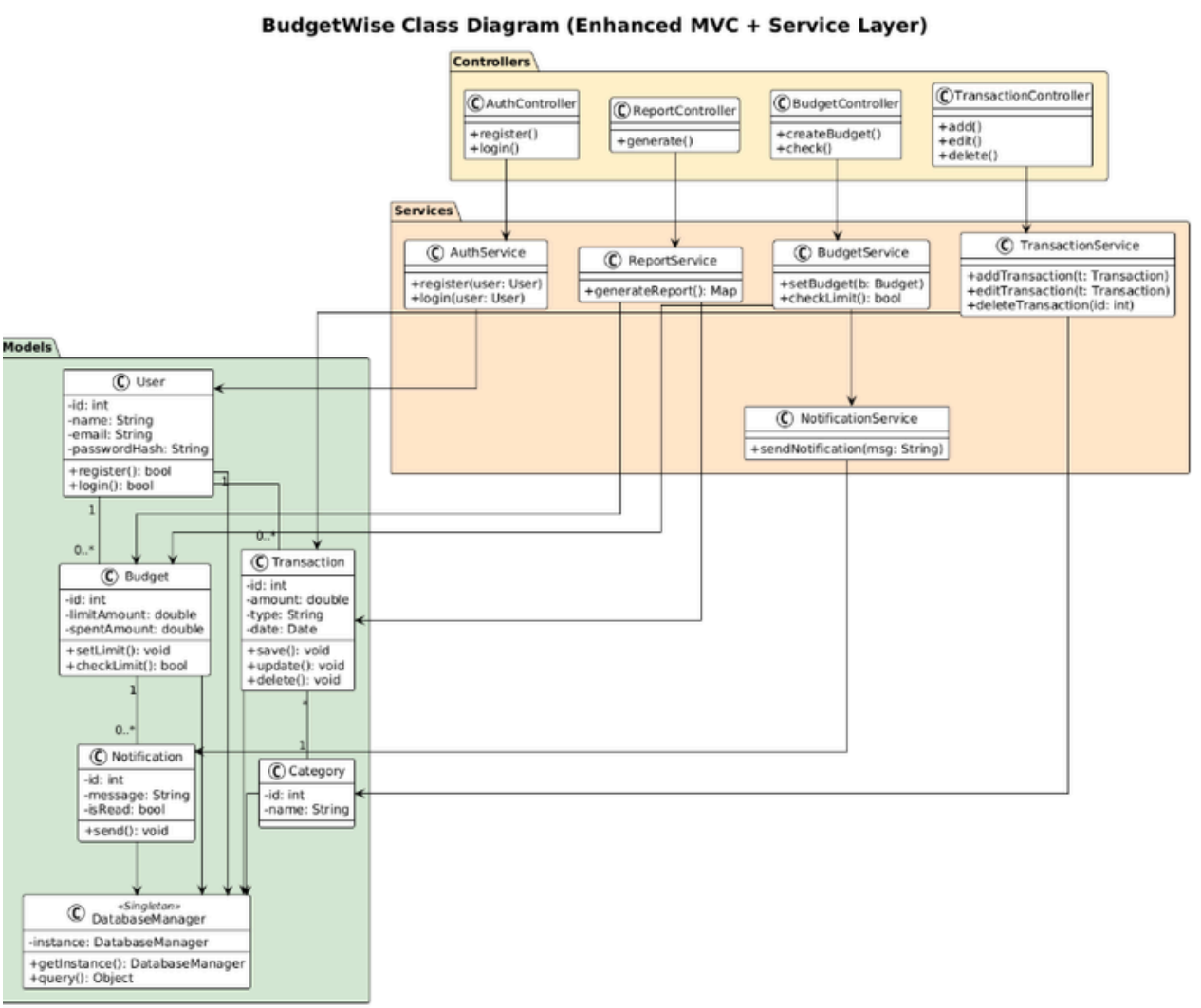


Figure A2: BudgetWise Class Diagram

PlantUML Code: See Section 2.5 — Code Block A2

2.3 Class Descriptions

ID	Class Name	Package	Description & Responsibility
C01	**User**	Model	Stores user credentials, profile info, and settings. Central entity linked to all financial data.
C02	**Transaction**	Model	Represents a single financial event (income or expense) with amount, category, date, and note.
C03	**Budget**	Model	Defines a spending limit for a specific category over a time period. Tracks usage vs. limit.
C04	**Goal**	Model	Represents a savings target with a name, target amount, deadline, and current saved amount.
C05	**Category**	Model	A classification label for transactions (e.g., Food, Transport). Can be predefined or custom.
C06	**Notification**	Model	Stores alert messages triggered by budget thresholds or goal milestones.
C07	**DatabaseManager**	Model	Singleton class that manages all DB connections and CRUD operations via SQLite/PostgreSQL.
C08	**AuthController**	Controller	Handles user registration, login validation, session management, and password hashing.
C09	**TransactionController**	Controller	Processes add/edit/delete/filter transaction requests. Triggers budget recalculations.
C10	**BudgetController**	Controller	Manages budget creation, editing, and real-time usage tracking. Sends alerts at thresholds.
C11	**GoalController**	Controller	Handles financial goal creation, contribution logging, and progress percentage calculation.
C12	**ReportController**	Controller	Aggregates transaction data and generates chart-ready datasets for the Report View.
C13	**LoginScreen**	View	Boundary class — renders the login/sign-up forms and sends credentials to AuthController.
C14	**DashboardScreen**	View	Boundary class — displays total balance, recent transactions, budget previews, and goal summaries.
C15	**TransactionScreen**	View	Boundary class — displays transaction history with filters; provides Add/Edit/Delete UI.
C16	**BudgetScreen**	View	Boundary class — shows budget cards with progress bars; provides Create/Edit budget UI.
C17	**GoalScreen**	View	Boundary class — lists all financial goals with progress; provides Add Goal / Contribute UI.
C18	**ReportScreen**	View	Boundary class — renders pie charts and bar charts using data from ReportController.

2.4 Sequence Diagrams

Seven sequence diagrams are provided, one for each of the most important user stories in the BudgetWise SRS. Each diagram shows the interaction between View, Controller, Model, and Database layers.

2.4.1 [SD-01] User Sign-Up (US #1)

Description: New user registers by providing name, email, and password. System validates, hashes the password, and creates the account.

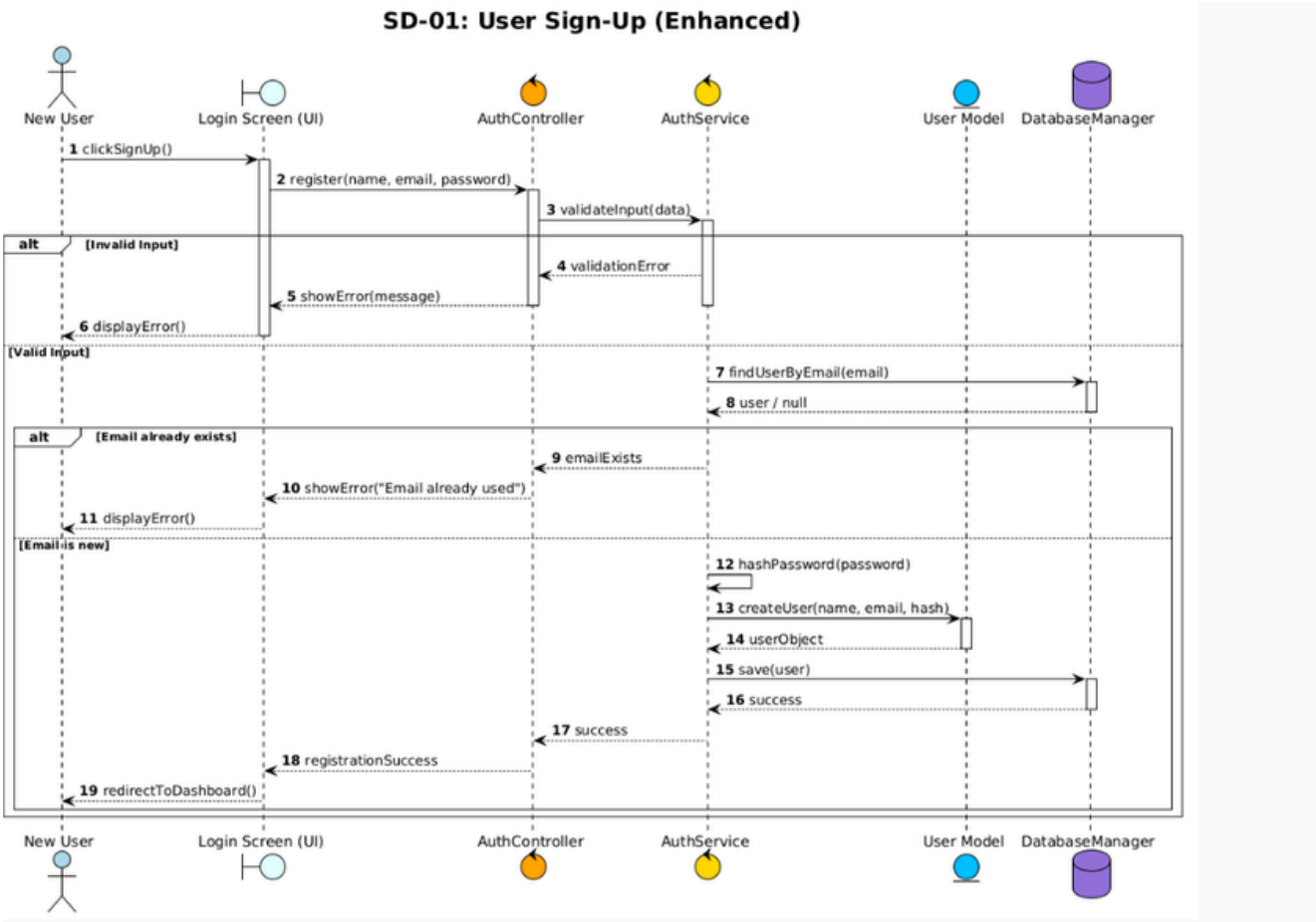


Figure SD-01: User Sign-Up (US #1)

PlantUML Code: See Section 2.5 — Code Block SD-01

2.4.2 [SD-02] User Log-In (US #2)

Description: Registered user authenticates using email and password. System verifies credentials and redirects to Dashboard.

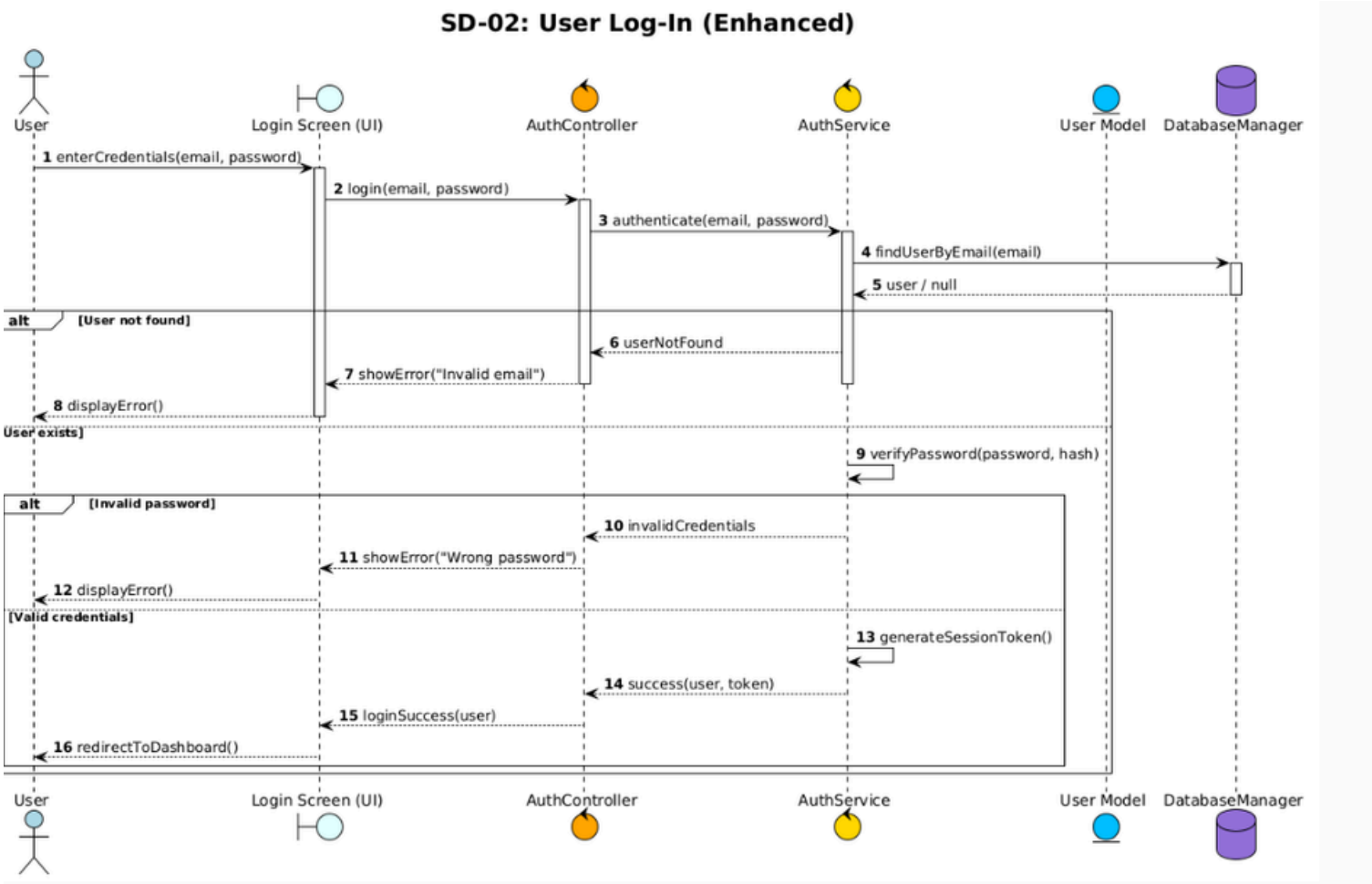


Figure SD-02: User Log-In (US #2)

PlantUML Code: See Section 2.5 — Code Block SD-02

2.4.3 [SD-03] Add Transaction (US #3)

Description: User adds a new income or expense transaction. System validates input, persists to DB, and updates Dashboard totals.

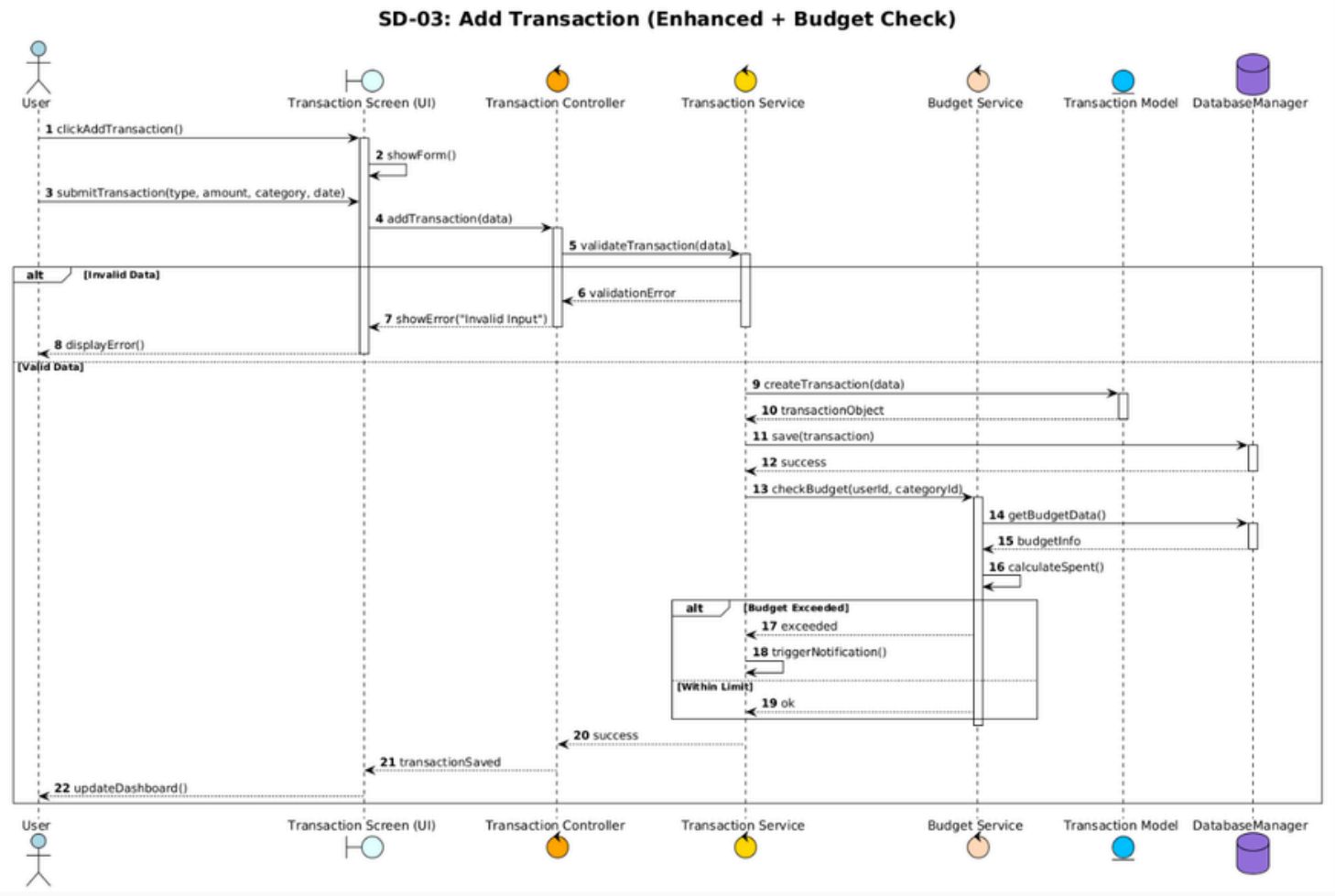


Figure SD-03: Add Transaction (US #3)

PlantUML Code: See Section 2.5 — Code Block SD-03

2.4.4 [SD-04] Create / Edit Budget (US #4)

Description: User creates a new budget for a spending category. System checks for duplicates, saves the budget, and begins tracking.

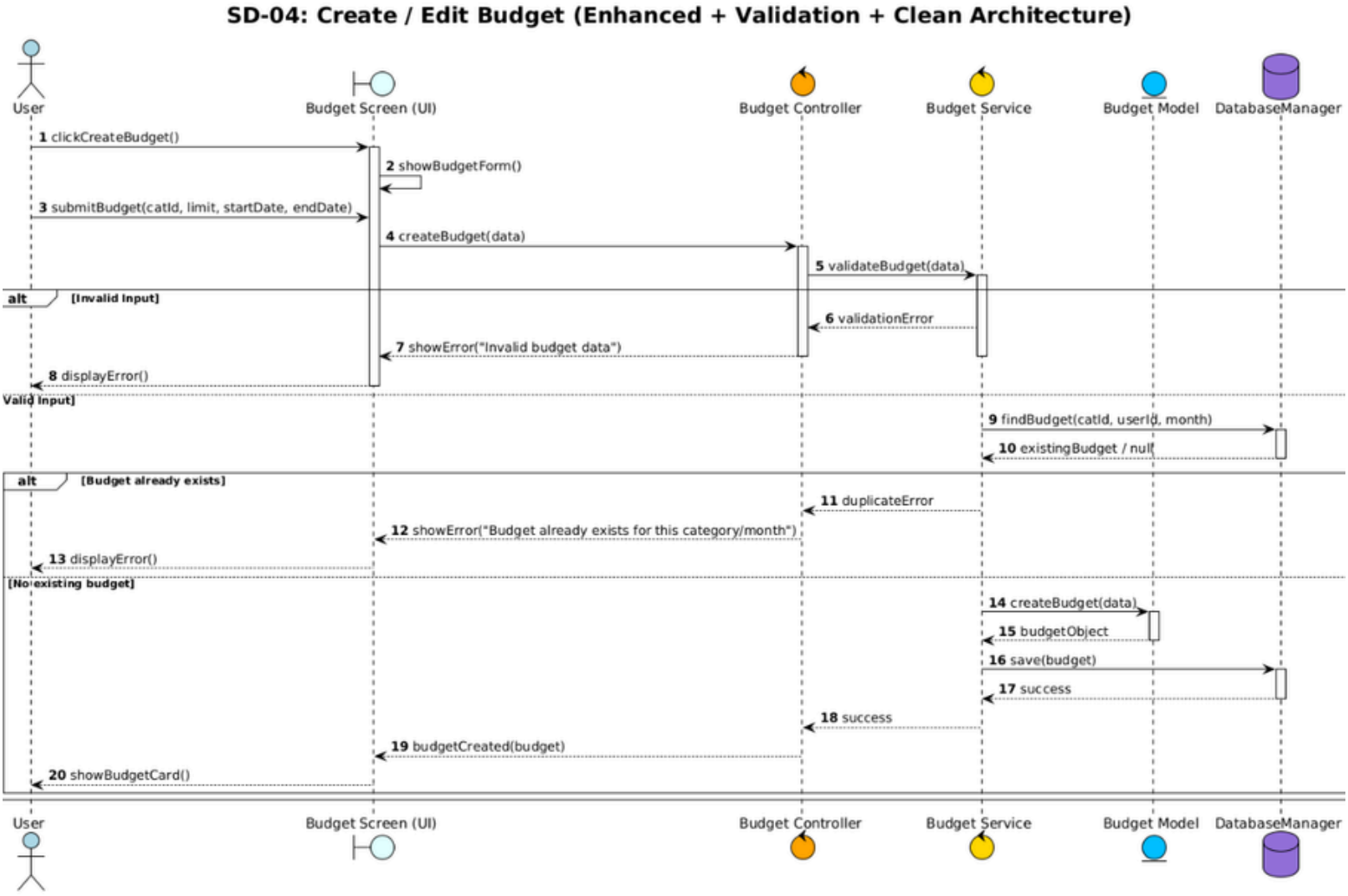


Figure SD-04: Create / Edit Budget (US #4)

PlantUML Code: See Section 2.5 — Code Block SD-04

2.4.5 [SD-05] Budget Over-Limit Alert (US #5)

Description: After logging an expense, system recalculates category spending. If threshold is reached, a notification is generated.

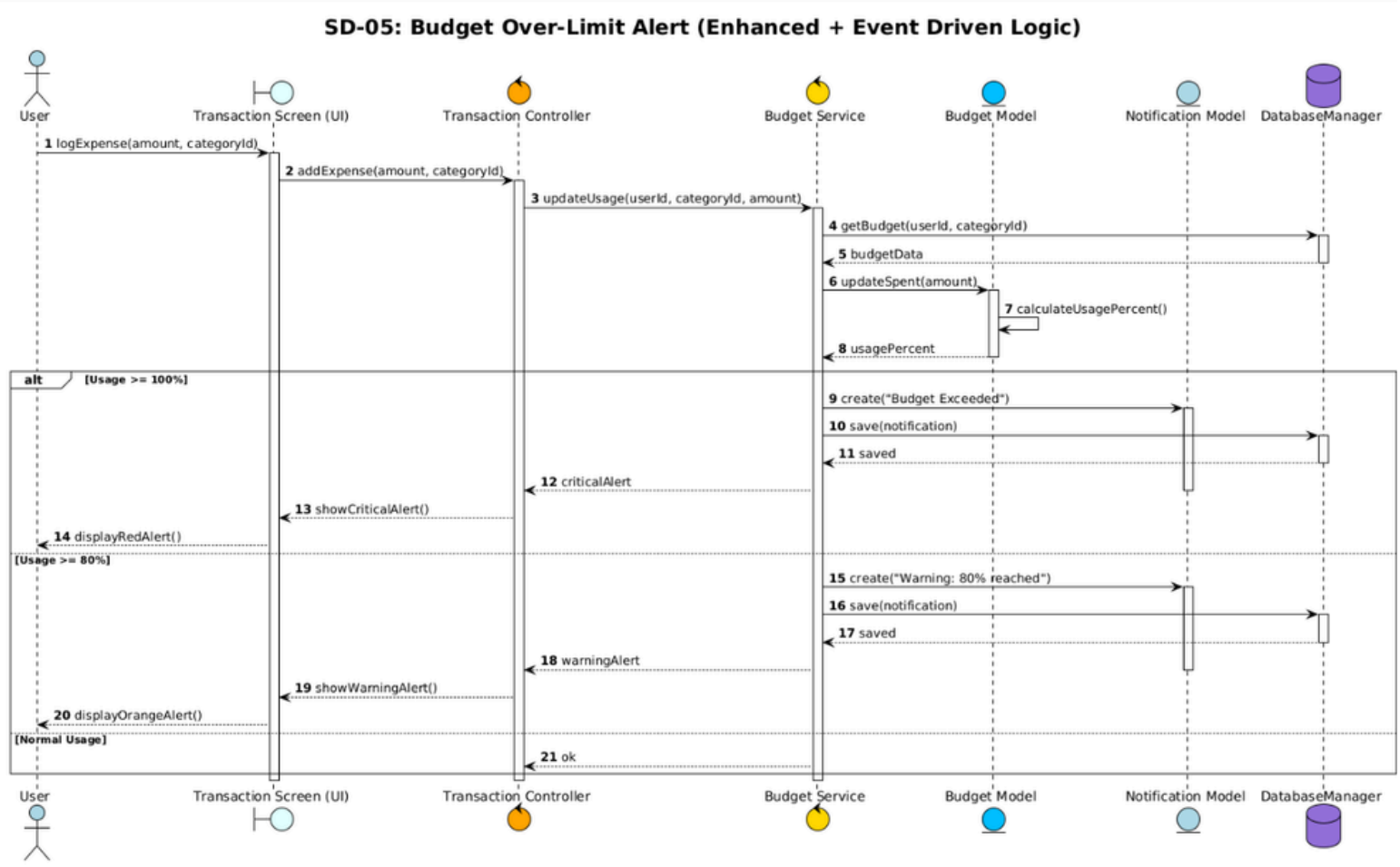


Figure SD-05: Budget Over-Limit Alert (US #5) PlantUML Code: See Section 2.5 — Code Block SD-05

2.4.6 [SD-06] Set and Track Financial Goal (US #6)

Description: User creates a savings goal with a target amount and deadline. System calculates required monthly savings and tracks progress.

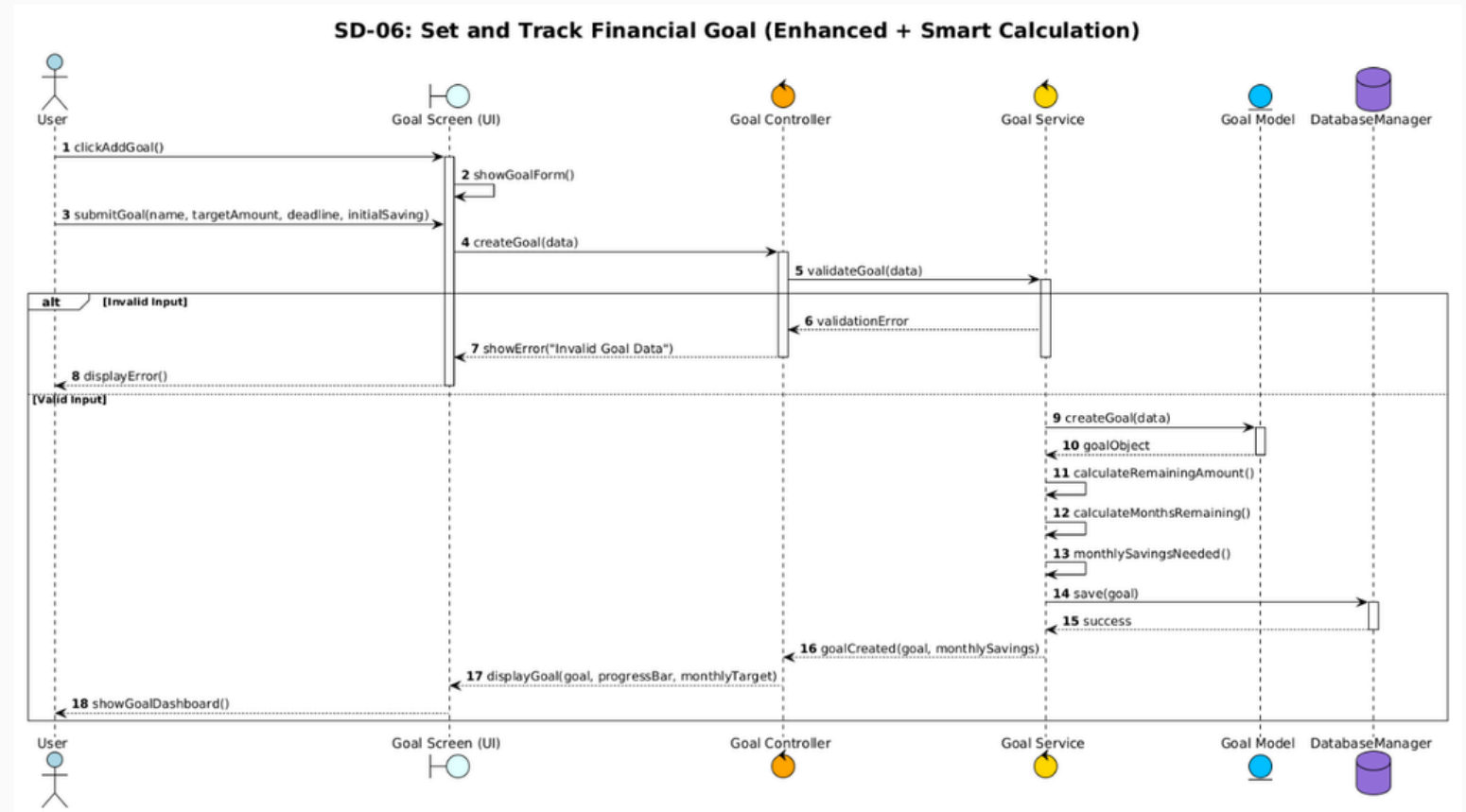


Figure SD-06: Set and Track Financial Goal

(US #6) PlantUML Code: See Section 2.5 —

Code Block SD-06

2.4.7 [SD-07] View Reports and Analytics (US #7)

Description: User navigates to Reports screen. System aggregates transaction data and renders pie and bar charts for the selected period.

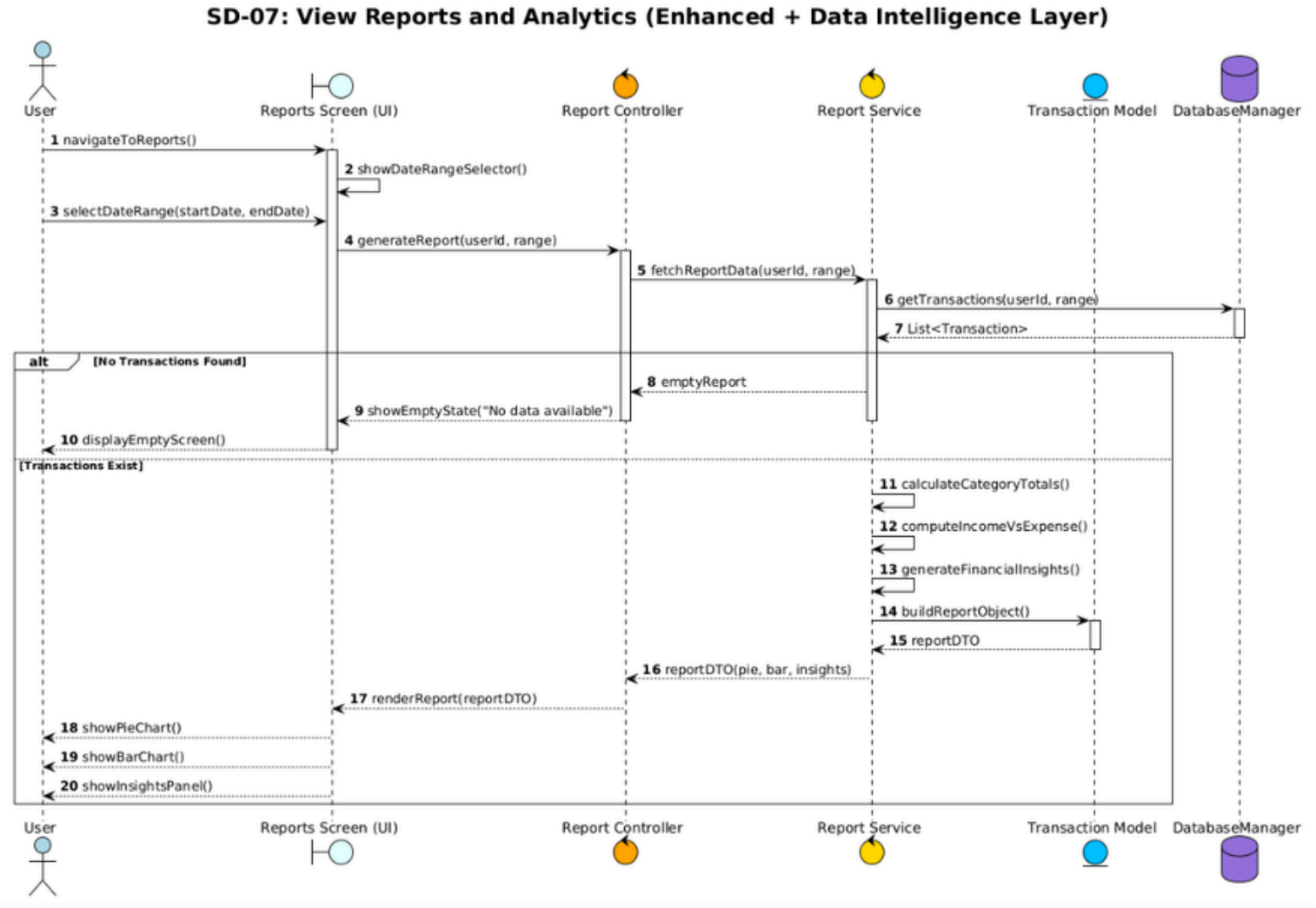


Figure SD-07: View Reports and Analytics

(US #7) PlantUML Code: See Section 2.5 —

Code Block SD-07

Class-Sequence Usage Traceability Table

Sequence Diagram	Classes Used	Key Methods
SD-01: User Sign-Up	LoginScreen, AuthController, User, DatabaseManager	register(), validateInput(), hashPassword(), save()
SD-02: User Log-In	LoginScreen, AuthController, User, DatabaseManager	login(), verifyPassword(), findByEmail()
SD-03: Add Transaction	TransactionScreen, TransactionController, Transaction, BudgetController, DatabaseManager	addTransaction(), validateInput(), checkThreshold(), save()
SD-04: Create Budget	BudgetScreen, BudgetController, Budget, DatabaseManager	createBudget(), findBudget(), save()
SD-05: Budget Alert	BudgetScreen, BudgetController, Budget, Notification, DatabaseManager	updateBudgetUsage(), getUsagePercent(), markRead(), save()
SD-06: Set Goal	GoalScreen, GoalController, Goal, DatabaseManager	createGoal(), calculateMonthlySavings(), addContribution(), save()
SD-07: View Reports	ReportScreen, ReportController, Transaction, DatabaseManager	generateReport(), aggregateCategoryTotals(), generateInsight()

Ownership Report

Item	Owner
Cover Page, Table of Contents, Document Structure	Ibrahem Ahmed Ezzat
Task 1 — Initiative, Job & Internship Research (Member 1 & 2)	Ibrahem Ahmed Ezzat, Mohamed Abdullah
Task 1 — Initiative, Job & Internship Research (Member 3 & 4)	Hesham Mohamed Fathy, Youssef Sayed Ali Ibrahem Ahmed Ezzat Mohamed
Architecture Diagram (Task 2)	Abdullah Abdulhafiz Hesham
Class Diagram + Class Descriptions (Task 2) Sequence	Mohamed Fathy Youssef Sayed
Diagrams SD-01, SD-02, SD-03 (Task 2) Sequence	Ali
Diagrams SD-04, SD-05, SD-06, SD-07 PlantUML Code	All members
+ Class-Sequence Table	

2.4 State Diagrams

State diagrams illustrate the different states an important object can occupy during its lifetime and the events/transitions that move it between those states. The **Budget** object was selected as the most representative domain entity, because its state directly drives the notification and alert subsystem.

2.4.1 Selected Object — Budget

Criterion	Justification
Multiple distinct states	A Budget moves through: Inactive → Active → Warning → Exceeded → Closed
Event-driven transitions	Each expense transaction is an event that may trigger a state change
System-wide impact	State changes propagate to NotificationService and DashboardScreen
Complexity	Includes guard conditions (usage ≥ 80 %, usage ≥ 100 %) and time-based expiry

2.4.2 Budget State Descriptions

State	Description	Entry Condition	Exit Condition
Inactive	Budget record exists but the tracking period has not started yet.	Budget is saved with a future startDate.	startDate is reached (system clock event).
Active	Budget is tracking spending; usage is below the warning threshold (< 80 %).	startDate reached OR budget is created with today's date	usage ≥ 0.80.
Warning	Spending has exceeded 80 % of the budget limit but not yet 100 %.	calculateUsagePercent() returns between 80 % and 100 %.	usage drops back below 80 % O
Exceeded	Spending has reached or surpassed the budget limit (≥ 100 %).	calculateUsagePercent() returns ≥ 100 %.	User resets / edits the limit, or endDate is reac
Closed	The budget period has ended (endDate passed). Read-only historical record.	System clock passes endDate.	Terminal state — no further transitions.

2.4.3 Transition Event Table

From State	Event / Guard	To State	Action
Inactive	startDate reached	Active	—
Active	addExpense() [usage ≥ 80 %]	Warning	NotificationService.create('Warning: 80% reached')
Warning	addExpense() [usage ≥ 100 %]	Exceeded	NotificationService.create('Budget Exceeded'); Dashboard.showRedAlert()
Warning	editBudget(newLimit) [usage drops < 80 %]	Active	Dashboard.clearWarning()
Exceeded	editBudget(newLimit) [usage < 80 %]	Active	Dashboard.clearAlert()
Exceeded	editBudget(newLimit) [80 ≤ usage < 100 %]	Warning	Dashboard.showOrangeAlert()
Active	endDate reached	Closed	Budget.markClosed()
Warning	endDate reached	Closed	Budget.markClosed()
Exceeded	endDate reached	Closed	Budget.markClosed()

2.4.4 Budget State Diagram

[SD-State-01: Budget State Diagram]

PlantUML / draw.io — render and paste here

Figure SD-State-01: Budget State Diagram — PlantUML source provided in Section 2.5 — Code Block SD-State-01

PlantUML hint — paste the following skeleton into plantuml.com to generate the diagram:

```
@startuml Budget_State_Diagram
skinparam state {
BackgroundColor #EAF3FB
BorderColor #2E74B5
ArrowColor #1F3864
FontName Helvetica
}
[*] --> Inactive : Budget created (future startDate)
Inactive --> Active : startDate reached
Active --> Warning : addExpense() [usage >= 80%]
Warning --> Active : editBudget() [usage < 80%]
Warning --> Exceeded : addExpense() [usage >= 100%]
Exceeded --> Active : editBudget() [usage < 80%]
Exceeded --> Warning : editBudget() [80% <= usage < 100%]
Active --> Closed : endDate reached
Warning --> Closed : endDate reached
Exceeded --> Closed : endDate reached
Closed --> [*]
@enduml
```

2.5 SOLID Principles & Design Patterns

This section demonstrates how the BudgetWise design consciously applies three SOLID principles and three Gang-of-Four Design Patterns. Each subsection names the principle/pattern, identifies the exact classes involved, and explains the benefit to the system.

2.5.1 SOLID Principles Applied

① Single Responsibility Principle (SRP)

"A class should have only one reason to change."

Aspect	Detail
Classes involved	AuthController · AuthService · User (Model) · DatabaseManager
How it is applied	Each class owns exactly one concern: • AuthController — receives HTTP/UI requests and delegates. • AuthService — validates credentials, hashes passwords, issues tokens. • User (Model) — stores and persists user data only. • DatabaseManager — executes raw SQL; knows nothing about business logic.
Benefit	Changing the hashing algorithm (e.g., bcrypt → argon2) only touches AuthService. Switching database
Evidence in design	Class Diagram (Figure A2) shows four separate classes with non-overlapping method sets for the auth

② Open/Closed Principle (OCP)

"Software entities should be open for extension but closed for modification."

Aspect	Detail
Classes involved	ReportController · ReportService · Transaction (Model)
How it is applied	ReportService exposes a stable generateReport(userId, range): Map interface. New report types (e.g.,
Benefit	Zero risk of regressions when new analytics features are added in future sprints.
Evidence in design	SD-07 (View Reports) shows ReportService calling calculateCategoryTotals(), computeIncomeVsExpe

③ Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules; both should depend on abstractions."

Aspect	Detail
Classes involved	All Service classes (AuthService, TransactionService, BudgetService, ReportService) · DatabaseMana
How it is applied	Every service class communicates with the persistence layer only through DatabaseManager's query()
Benefit	Testability: services can be unit-tested using a mock DatabaseManager with no real DB connection re
Evidence in design	Architecture Diagram (Figure A1) shows a single Data Layer arrow from all service classes pointing to

SOLID Summary

Principle	Applied In	Key Benefit
Single Responsibility (SRP)	Auth subsystem (4 classes)	Independent changeability
Open / Closed (OCP)	ReportService extension points	Safe feature additions
Dependency Inversion (DIP)	All Services → DatabaseManager	Testability & DB swap

2.5.2 Design Patterns Applied

① Singleton Pattern — DatabaseManager

Aspect	Detail
Category	Creational
Intent	Ensure a class has only one instance and provide a global access point.
Class	DatabaseManager
Implementation	DatabaseManager declares a private static instance field and exposes getInstance(): DatabaseManager a
Why needed	Opening multiple simultaneous DB connections in a mobile/desktop app causes concurrency errors and r
Evidence	Class Diagram (Figure A2) shows «Singleton» stereotype on DatabaseManager.

② Observer Pattern — Budget Alert / Notification Subsystem

Aspect	Detail
Category	Behavioural
Intent	Define a one-to-many dependency so that when one object changes state, all dependents are notified au
Classes	Subject: BudgetService Observers: NotificationService, DashboardScreen, TransactionScreen
Implementation	When BudgetService.updateUsage() detects usage ≥ 80 % or ≥ 100 %, it calls NotificationService.sendN
Why needed	Multiple UI screens and the notification log all need to react to a single budget state change. Without Obs
Evidence	SD-05 (Budget Over-Limit Alert) shows BudgetService firing events received by both the Transaction Scr

③ MVC Architectural Pattern — Entire System

Aspect	Detail
Category	Architectural / Structural
Intent	Separate an application into three interconnected layers to decouple data (Model), business logic (Contro
Classes	View: LoginScreen, DashboardScreen, TransactionScreen, BudgetScreen, GoalScreen, ReportScreen Controller: AuthController, TransactionController, BudgetController, GoalController, ReportController Model: User, Transaction, Budget, Goal, Category, Notification, DatabaseManager
Implementation	Every user action originates in a View (e.g., BudgetScreen.clickCreateBudget()). The View calls its paired
Why needed	Enables independent development and testing of UI, logic, and data layers. All seven sequence diagrams
Evidence	Architecture Diagram (Figure A1) and all Sequence Diagrams.

Design Patterns Summary

Pattern	Category	Applied To	Benefit
Singleton	Creational	DatabaseManager	Single DB connection; no resource waste
Observer	Behavioural	BudgetService → Notification/UI	Decoupled multi-screen alert propagation

Pattern	Category	Applied To	Benefit
MVC	Architectural	Entire BudgetWise system	Separation of concerns; testability

All three SOLID principles and three design patterns listed above are directly traceable to the class diagram (Figure A2) and the sequence diagrams (Figures SD-01 through SD-07) presented in the preceding sections.